Agenda

1.	Einleitung	2
2.	Grundlagen	19
3.	Programmiersprachen	31
4.	Automaten	45
5.	Grammatiken	109
6.	Syntaxanalyse	150

Beispiel

$S \rightarrow ABC$,
$A \rightarrow aa \mid b$,
$B \rightarrow bbB \mid \varepsilon$,
$C \rightarrow c$

Aufgabe: Leiten Sie das Wort *aabbc* ab.

Wie sind Sie dabei vorgegangen?

$Lösung$: $S \Rightarrow ABC \Rightarrow aaBC \Rightarrow aabbBC \Rightarrow aabbC \Rightarrow aabbc$

Wie würden Sie einen Parser für diese Grammatik programmieren?

Top-Down Parser mit rekursivem Abstieg

$$S \rightarrow ABC\,, A \rightarrow aa \mid b, B \rightarrow bbB \mid \varepsilon, C \rightarrow c$$

```
void S(); void A(); void B(); void C();

char token;

void parser() {
   scanner(); // Vorausschau-Token lesen
   S();       // Eingabe analysieren
}

void S() {
   regel("S -> ABC");
   A(); B(); C();
}

void A() {
  switch(token) {
    case 'a':
      regel("A -> aa"); check('a'); check('a');
     break;
   case 'b':
     regel("A -> b"); check('b');
     break;
   default:
     fehler("a oder b erwartet");
   }
}
```

```
void B() {
 if (token=='b') {
   regel("B -> bbB"); check('b'); check('b'); B();
 }
 else
   regel("B -> epsilon");
}

void C() {
   regel("C -> c"); check('c');
}

void regel(char *s){
  printf("Regel: %s\n", s);
}

void check(char erwartetes_token){
  if ( erwartetes_token != token ){
     fehler(„Unerwartetes Token„);
     exit(1);
   }
  else scanner();
}

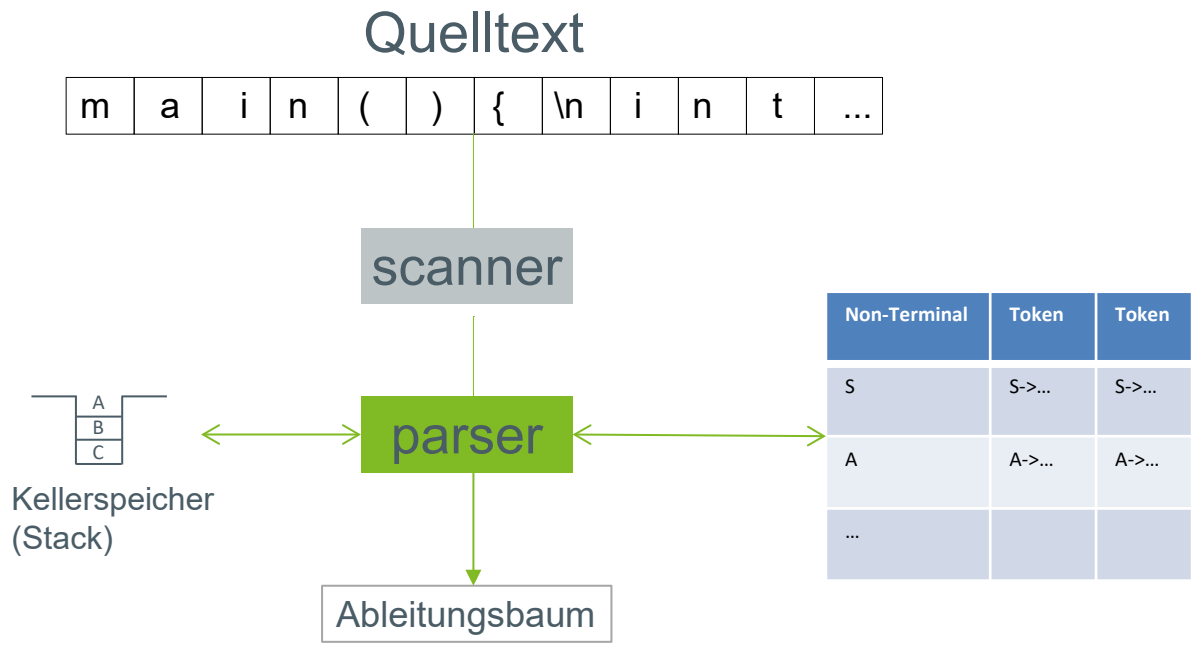
int main() {
  parser();
}
```

LL(k) Grammatik

Falls eine Grammatik G mit einem Lookahead k im Top-Down Parser Algorithmus eine eindeutige Regelauswahl erlaubt, dann heißt G **LL(k) Grammatik**.

- L : Lesen von links nach rechts.
- L : Satzformen von links nach rechts abarbeiten.

Alternativ: Tabellengesteuerter Top-down Parser



Der Parser wird genau wie der Scanner durch Tools generiert.

Tabellengesteuerter Top-down Parser

Beispiel:

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow *\ F\ T' \mid \varepsilon$

$F \rightarrow (\ E\ ) \mid x$

w = (x + x * (x + x))

Woher „weiß“ die Tabelle, dass ein (das richtige Token für E ist?

Zwei mögliche Fälle



In beiden Fällen kann der Parser, wenn er t liest, nicht entscheiden, welche der beiden Regeln $A \to \alpha$ oder $A \to \alpha'$ er anwenden soll.

First-Mengen

Definition: Sei $\alpha \in (\mathrm{T} \cup N)^*$, d.h. eine Satzfolge aus Terminalen und Nichtterminalen, dann sei $\boldsymbol{First(\alpha)}$ die Menge der führenden Terminalzeichen, die sich aus α ableiten lassen.

$$First(\alpha) = \{\, t \in (\mathrm{T} \cup \{\epsilon\}) \mid \alpha \overset{*}{\Rightarrow} t\,\beta \;\; und \;\; (t \in T \; oder \; t\beta = \epsilon)\}$$

Offensichtlich:

i. Wenn $a \in \mathrm{T}$, dann ist $First(a) = \{\mathrm{a}\}$
ii. Wenn $(A \rightarrow \epsilon) \in P$, dann ist $\epsilon \in First(A)$
iii. Wenn $(A \rightarrow a\beta) \in P$, dann ist $a \in First(A)$
iv. Wenn $\mathrm{A} \in N \; und \; A \rightarrow Y_1 \ldots Y_k$ und $\epsilon \in First(Y_1)$ und $\ldots \epsilon \in First(Y_k)$, dann ist $\epsilon \in First(A)$ $\quad (Y_i \in (\mathrm{T} \cup N))$
v. Wenn $\mathrm{A} \in N \; und \; A \rightarrow Y_1 \ldots Y_k$ und $\epsilon \in First(Y_1)$ und $\ldots \epsilon \in First(Y_{i-1}) \; mit$ $1 \leq i \leq k \; und \; \epsilon \notin First(Y_i)$, dann ist $First(Y_i) \subseteq First(A)$

Algorithmus zu Berechnung von First-Mengen

(I) $Für\ alle\ A \in N: First(A) := \begin{cases} \{\epsilon\}, falls\ (A \to \epsilon) \in P \\ \{a\}, falls\ A \to a\beta \in P \end{cases}$

(II) Wiederhole für alle Produktionen $A \to Y_1 \ldots Y_k$:

Für alle i mit 1<=i<=k :

$First(A) := First(A) \cup (First(Y_i) \backslash \{\epsilon\})$, *solange bis* $\epsilon \notin First(Y_i)$.

$First\ (A) := First(A) \cup \{\epsilon\}, falls\ \epsilon \in First(Y_1)\ und\ \ldots \epsilon \in First(Y_k)$

Aufgabe

Berechnen Sie die First-Mengen für die folgende Grammatik:

S → Aa

A → b | cB | ε

B → bb

(I) First(B) := {b}
First(A) := {b, c, ε}
First(S) := ∅

(II) Für die Regel S → Aa ergibt sich für i=1 und i=2:

$$\text{First}(S) := First(S) \cup (First(A) \setminus \{\epsilon\}) = \{b, c\}\ und$$
$$First(S) := First(S) \cup First(a) = \{b, c\} \cup \{a\} = \{a, b, c\}$$

Follow-Mengen

Definition: **Follow(A)** ist die Menge aller Terminale, die rechts von A in einer Satzform vorkommen können.

$$Follow(A) = \begin{cases} \{t \in \mathrm{T}\}, & \text{falls } S \overset{*}{\Rightarrow} \beta A t \gamma \ \ und\ \beta, \gamma \in (\mathrm{T} \cup N)^* \\ \{\$\}, & falls\ S \overset{*}{\Rightarrow} \beta A \quad und\ \beta \in (T \cup N)^* \end{cases}$$

Algorithmus:

$Follow(S) = \{\$\}$

Für alle $\mathrm{A} \in N \setminus \{S\}$: $Follow(A) := \emptyset$

Für alle $\mathrm{A} \in N$:

  Für alle $B \rightarrow \alpha A \beta \in P$: $Follow(A) := Follow(A) \cup (First(\beta) \setminus \{\epsilon\})$

Wiederhole

  Für alle $\mathrm{A} \in N$:

    Für alle $B \rightarrow \alpha A \beta \in P\ mit\ \epsilon \in First(\beta)$

      $Follow(A) := Follow(A) \cup Follow(B)$

Solange, bis sich an den Follow-Mengen nichts mehr ändert.

Aufgabe

Berechnen Sie die Follow-Mengen für die Grammatik:

S → Aa
A → b | cB | ε
B → bb

Follow(S)={$}, Follow(A)=∅, Follow(B)=∅

1. Schleife: Follow(A):=First(a) = {a} (wegen Regel S → Aa)
2. Schleife: Follow(B):=Follow(A) = {a} (wegen Regel A → cB)

Also: Follow(S)={$},
Follow(A)=Follow(B) = {a}

Parser-Tabelle

Zeilen: Nichtterminalsymbole A

Spalten: Terminalsymbole x

Eintrag **TAB(A, x)** ist die Regel, die auf A angewandt werden soll, wenn der Lookahead x ist.

Algorithmus:

Für alle $A \rightarrow \alpha \in P$:

Für alle Terminale $a \in First(\alpha)$:

$$\mathrm{TAB}(\mathrm{A}, a) = A \rightarrow \alpha.$$

Falls $\epsilon \in \mathrm{First}(\alpha)$, dann setze für jedes Terminal $\mathrm{b} \in \mathrm{Follow}(\mathrm{A})$:

$$\mathrm{TAB}(\mathrm{A}, b) = A \rightarrow \alpha.$$

Alle leeren Tabelleneinträge werden als error definiert.

Beispiel

S → aA
A → B | cAd | ε
B → bb

=>
First(A) = {b, c, ε},
First(B) = {b},
First(S) = {a}

Follow(S) = {$},
Follow(A) = {d, $}
Follow(B) = {d, $}

S → aA
First(aA)={a}
TAB(S, a) = S → aA

A → B
First(B)={b}
TAB(A, b)= A → B

A → cAd
First(cAd)={c}
TAB(A,c)=A → cAd

A → ε
First(ε) = {ε}
Follow(A)={d, $}
TAB(A, d) = A → ε und TAB(A, $)=A → ε

B → bb
First(bb)={b}
TAB(B, b) = B → bb

	a	b	c	d	$
S	S→aA				
A		A→B	A→cAd	A→ε	A→ε
B		B→bb			

Praktikumsaufgabe 2

1. Implementieren Sie die Algorithmen zur Berechnung der First- und der Follow-Mengen, sowie den Algorithmus zur Berechnung der LL(1)-Parsertabelle.

   → Siehe Hinweise in der Praktikumsaufgabe!

2. Erstellen Sie eine Grammatik für TiEL.

   → Beginnen Sie am besten mit einer EBNF, diskutieren Sie diese mit den Betreuern im Praktikum.
   In der nächsten VL wird erklärt, wie man die EBNF in Code umwandelt.

Hinweise zur Definition einer EBNF für TiEL

1. Ein TiEL-Programm besteht aus einer Liste von *Statements*, die möglicherweise leer ist.
2. Ein *Statement* ist ein *IfStatement*, ein *Block*, ein *Expressionstatement*, eine *Funktion*, …
3. Ein IfStatement besteht aus dem Schlüsselwort „if“, linke Klammer, einem Ausdruck, rechte Klammer und einem Block, optional: „else“ gefolgt von Block.
4. Eine Expression besteht auf der untersten Ebene aus Zahlen, true, false, nil, …
5. Eine Expression1 ist Expression2 oder Expression1 +/- Expression2.

1. *Programm = { Statements }.*
2. *Statement = IfStmt | Block | ExprStmt | Function |… .*
3. *IfStmt* = "if" "("*Expr* ")" *Block ["else" Block].*
4. *Expr3 = Number | "true" | "false" | …*
5. *Expr1 = Expr2 | Expr1 ("+"|"-") Expr2.*
   …

LL(1) Grammatik

Eine kontextfreie Grammatik ist eine LL(1)-Grammatik genau dann, wenn aus

i. $S \overset{*}{\Rightarrow} x\,A\,\beta \rightarrow x\alpha\beta \overset{*}{\Rightarrow} xy$, und

ii. $S \overset{*}{\Rightarrow} x\,A\,\beta \rightarrow x\alpha'\beta \overset{*}{\Rightarrow} xy'$, und

iii. $First(y) = First(y')$

immer $y = y'$ folgt.

D.h., an jeder Stelle einer Ableitung kann man mit dem ersten Terminalzeichen eines aus $A\ \beta$ ableitbaren Terminalwortes eindeutig die auf A anzuwendende Regel $A \rightarrow \alpha$ bestimmen.

Oder: Für eine LL(1) Grammatik kann man die Parsertabelle eindeutig bestimmen.

Schlussfolgerungen über LL(1)

- Wenn es zwei Regeln $A \to \alpha \; und \; A \to \alpha' \; mit \; \alpha \neq \alpha' \; gibt$, dann ist G LL(1) genau dann, wenn $First\big(\alpha Follow(A)\big) \cap First\big(\alpha' Follow(A)\big) = \emptyset$.

- Für jede Grammatik ist entscheidbar, ob sie LL(1) ist oder nicht.

- Jede LL(1) Grammatik ist eindeutig.
  (Eine Grammatik heißt mehrdeutig, wenn es ein Wort mit zwei verschiedenen Linksableitung gibt).

- LL(1) Grammatiken sind nicht linksrekursiv!

- Nicht jede kontext-freie Sprache lässt sich durch eine LL(k)-Grammatik darstellen!

Top-Down Analyse für LL(1)-Grammatiken

```
push(S)              /* Startsymbol S auf Stack              */
a:=nexttoken()       /* Eingabesymbol := erstes Symbol */
repeat
        if Stack leer and a=EOF
        then accept                    /* erfolgreich beendet*/
        else X:= oberstes Stackelement
            if X = a then              /* ”Match” eines Terminalsymbols */
                pop()
                a:=nexttoken()
            elseif ( X ∈ N and TAB[X, a] = (X → Y_1 ... Y_k) ) then
                print(X → Y_1 ... Y_k)   /* Ausgabe des nächsten Ableitungsschritts*/
                pop()
                push(Y_k) , . . . , push(Y_1)
                else
                        Fehlerbehandlung
        end if end if
end repeat
```

Beispiel

Eingabe: accbbdd

$S \rightarrow aA$
$A \rightarrow B \mid cAd \mid \varepsilon$
$B \rightarrow bb$

	a	b	c	d	$
S	S→aA				
A		A→B	A→cAd	A→ε	A→ε
B		B→bb			

Stack	restliche Eingabe	Aktion
S	accbbdd	S → aA
aA	accbbdd	pop
A	ccbbdd	A→cAd
cAd	ccbbdd	pop
Ad	cbbdd	A→cAd
cAdd	cbbdd	pop
Add	bbdd	A→B
Bdd	bbdd	B→bb
bbdd	bbdd	pop
bdd	bdd	pop
dd	dd	pop
d	d	pop
	$	accepted

Schlussfolgerungen über den Top-Down Parser

- Wenn G eine LL(1)-Grammatik ist und $w \in \mathrm{T}^*$, dann gilt:
  - (i) Der Algorithmus (siehe Vorseite) terminiert
  - (ii) Wenn der Algorithmus angewandt auf w mit leerem Keller und vollständig gelesenem Wort w terminiert, dann ist $w \in L(G)$
  - (iii) Wenn der Algorithmus w nicht akzeptiert, dann ist $w \notin L(G)$

– Der Algorithmus benötigt $\mathcal{O}(|w|)$ Schritte, um das Wort w zu analysieren.